

学号：22920202200773

姓名：孟媛

实验四 直接三角分解法求解线性方程组

一、实验目的

使用直接三角分解法求解线性方程组。

二、数值计算原理

求解 $Ax = b$ 时，将 A 分解为 $L \times U$ 的形式，其中， L 为下三角矩阵，对角线元素均为

1， u 为上三角元素。这样可以根据 $\begin{cases} ly = b \\ ux = y \end{cases}$ 来计算 $Ax = b$

根据两个矩阵相等，所有对应元素均相等的特点，可以列方程，最终求出 L 和 u 中所有元素，即利用 $u_{1j} = a_{1j} (j = 1, 2 \dots n)$ ， $l_{i1} = a_{i1}/u_{11}$ ， $u_{kj} = a_{kj} - \sum_{r=1}^{k-1} l_{kr} * u_{rj} (j = k, k+1 \dots n)$ ， $l_{ik} = (a_{ik} - \sum_{r=1}^{k-1} l_{ir} * u_{rk})/u_{kk} (i = k+1, k+2 \dots n)$ 来求出 L 和 u 之后进行回代，解出 $\begin{cases} ly = b \\ ux = y \end{cases}$ 就可以求出原方程组。

三、源程序介绍

矩阵处理函数上传 1 个参数：方程组 $Ax = b$ 的 A 矩阵。

求解函数上传 2 个参数：方程组 $Ax = b$ 的 A 矩阵的 L 和 U ，以及 b

计算过程：

(1) 初始化上三角矩阵 u ，和下三角矩阵 L 。

(2) 根据公式 $u_{1j} = a_{1j} (j = 1, 2 \dots n)$ 和 $l_{i1} = a_{i1}/u_{11}$ 计算出 U 第一行和 L 第一列元素。

(3) 根据公式 $u_{kj} = a_{kj} - \sum_{r=1}^{k-1} l_{kr} * u_{rj} (j = k, k+1 \dots n)$ 和 $l_{ik} = (a_{ik} - \sum_{r=1}^{k-1} l_{ir} * u_{rk})/u_{kk} (i = k+1, k+2 \dots n)$ 分别计算出上三角矩阵和下三角矩阵。

(4) 回代：根据

$$\begin{cases} y_1 = b_1 \\ y_k = b_k - \sum_{j=1}^{k-1} l_{kj} y_j \quad (k=2,3,\Lambda n) \end{cases} \text{ 和 } \begin{cases} x_n = y_n / u_{nn} \\ x_k = (y_k - \sum_{j=k+1}^n u_{kj} x_j) / u_{kk} \quad (k=n-1, n-2, \Lambda, 1) \end{cases}$$

计算出 y 和 x。

```

1  #三角法求解线性方程组
2  import numpy
3  from scipy.optimize import fsolve
4
5  #处理矩阵
6  def matrix_deal(matrix):
7      l = len(matrix)
8      # 创建两个大小与原矩阵相同的全零矩阵
9      Matrix_L = numpy.zeros((l, l), dtype = float)
10     Matrix_U = numpy.zeros((l, l), dtype = float)
11     # 将左矩阵对角线元素变为零 将右矩阵首行元素赋值 左矩阵首列赋值
12     for i in range(l):
13         Matrix_L[i][i] = 1
14         Matrix_U[0][i] = matrix[0][i]
15         Matrix_L[i][0] = matrix[i][0] / Matrix_U[0][0]
16     for i in range(1, l):
17         # 对右矩阵的第i行赋值
18         Matrix_U[i, i:] = matrix[i, i:] - \
19             numpy.sum(Matrix_L[i, :i].reshape(i, 1) * Matrix_U[:i, i:], axis = 0)
20         # 对左矩阵的第i列赋值
21         if i == l - 1:
22             continue
23         Matrix_L[i + 1:, i] = (matrix[i + 1:, i] - numpy.sum(Matrix_L[i + 1:, :i]
24             * Matrix_U[:i, i].reshape(1, i), axis = 0)) / Matrix_U[i][i]
25     # 返回左右矩阵
26     return Matrix_L, Matrix_U
27
28 #计算线性方程组的解
29 def answer(Matrix_L, Matrix_U, matrix_an):
30     # 求得 L * y = B 的解
31     Matrix_L = numpy.hstack([Matrix_L, matrix_an])
32     m, n = Matrix_L.shape
33     for i in range(n - 1):
34         Matrix_L[i + 1:, :] = Matrix_L[i + 1:, :] - \
35             Matrix_L[i] * Matrix_L[i + 1:, i].reshape(m - i - 1, 1)
36     # 求得 U * x = y 的解
37     matrix_an = Matrix_L[:, -1].reshape(m, 1)
38     Matrix_U = numpy.hstack([Matrix_U, matrix_an])
39     for i in range(m - 1):
40         i = m - i - 1
41         Matrix_U[:, i] = Matrix_U[:, i] - Matrix_U[i] * \
42             Matrix_U[:, i].reshape(i, 1) / Matrix_U[i][i]
43     for i in range(m):
44         Matrix_U[i][-1] = Matrix_U[i][-1] / Matrix_U[i][i]
45         Matrix_U[i][i] = 1
46     return Matrix_U[:, -1].reshape(m, 1)
47
48 if __name__ == '__main__':
49     A = numpy.array([2, 1, 5, 4, 1, 12, -2, -4, 5],
50                     dtype = float).reshape(3, 3)
51     b = numpy.array([11, 27, 12], dtype=float).reshape(3, 1)
52     print('原增广矩阵为:')
53     print(numpy.hstack([A, b]))
54     Matrix_L, Matrix_U = matrix_deal(A)
55     matrix_a = answer(Matrix_L, Matrix_U, b)
56     print('L为:')
57     print(Matrix_L)
58     print('U为:')
59     print(Matrix_U)
60     print('自编函数求得的解集为:')
61     print(matrix_a)
62     print("自带函数求得的解集为:\n", numpy.linalg.solve(A, b))
63

```

四、程序执行情况

4.1 书上给的例子:

```
A = numpy.array([2, 1, 5, 4, 1, 12, -2, -4, 5],
                 dtype = float).reshape(3, 3)
b = numpy.array([11, 27, 12], dtype=float).reshape(3, 1)
```

原增广矩阵为:

```
[[ 2.  1.  5. 11.]
 [ 4.  1. 12. 27.]
 [-2. -4.  5. 12.]]
```

L为:

```
[[ 1.  0.  0.]
 [ 2.  1.  0.]
 [-1.  3.  1.]]
```

U为:

```
[[ 2.  1.  5.]
 [ 0. -1.  2.]
 [ 0.  0.  4.]]
```

自编函数求得的解集为:

```
[[ 1.]
 [-1.]
 [ 2.]]
```

自带函数求得的解集为:

```
[[ 1.]
 [-1.]
 [ 2.]]
```

PS D:\myy\Math_Analysis> □

4.2 书上未给的例子

```
A = numpy.array([2, 3, 10, 5, 1, 1, 6, 2, 5, 1, 3, 2, 1, 1, 3, 4],
                 dtype = float).reshape(4, 4)
b = numpy.array([2, 1, -3, -3])
```

```
原增广矩阵为：
[[ 2.  2.  3.  3.]
 [ 4.  7.  7.  1.]
 [-2.  4.  5. -7.]]
L为：
[[ 1.  0.  0.]
 [ 2.  1.  0.]
 [-1.  2.  1.]]
U为：
[[2.  2.  3.]
 [0.  3.  1.]
 [0.  0.  6.]]
自编函数求得的解集为：
[[ 2.]
 [-2.]
 [ 1.]]
自带函数求得的解集为：
[[ 2.]
 [-2.]
 [ 1.]]
PS D:\myy\Math_Analysis>
```

五、结果分析

使用直接三角分解法解线性方程组与自带函数计算结果一致。直接三角分解法名字很直接，但计算过程很繁琐，但是也是解方程组的好方法。

六、实验体会

此次实验，我认识到了直接三角分解法的优秀性能，同时学习完成了自编函数的代码。我对直接三角分解法的理解更加深入，同时体会到了它和高斯列主元消去法的差异，它们都是比较好的方法。